A Capstone Project Report

On

RentAPlace

Submitted by:

Prashant Gautam

Technology:

Angular + ASP.NET Core Web API

Date:

23 March 2026

Instructor:

Ramesh Nediyadath

DECLARATION

I, Prashant Gautam, hereby declare that the project titled "RentAPlace – Property Booking Application" submitted as part of the training program conducted by Great Learning for Wipro is my original work.

I confirm that this project has been developed by me and has not been submitted previously for any academic or professional purpose. All sources of information used in this project have been properly acknowledged.

I take full responsibility for the authenticity and correctness of the work presented in this report.

Date: 23 March 2026
Place: Samastipur, Bihar

Signature: ____
(Prashant Gautam)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Wipro and Great Learning for providing me with the opportunity to work on this capstone project titled “RentAPlace”.

I would also like to thank the instructors and mentors for their continuous guidance, support, and valuable feedback throughout the training program. Their guidance helped me enhance my technical skills in full-stack development using Angular and ASP.NET Core.

Finally, I am thankful for the opportunity to apply my knowledge in a practical environment and gain hands-on experience in building a real-world application.

Date: 23 March 2026
Signature: ____
(Prashant Gautam)

ABSTARCT

The increasing reliance on digital platforms has transformed how people search for and access rental properties. Traditional methods of finding accommodations are often time-consuming, less transparent, and dependent on intermediaries. To address these challenges, the **RentAPlace** project is developed as a web-based application that simplifies the process of property renting by connecting property owners and renters on a single platform.

The primary aim of this project is to design and develop a full-stack web application that allows users to search, view, and reserve properties based on their preferences such as location, price, and available amenities. The system provides an intuitive interface where users can explore property listings, view images, and communicate directly with property owners. It also includes features such as user authentication, property filtering, reservation management, and messaging to ensure a smooth and efficient user experience.

From the owner's perspective, the application enables effective management of property listings through functionalities like adding, updating, and deleting properties. Owners can also respond to user inquiries and manage reservation requests, making the system interactive and dynamic. The inclusion of image upload and notification features further enhances usability and engagement.

The application is built using Angular for the frontend and ASP.NET Core Web API for the backend, ensuring a scalable and responsive architecture. Entity Framework is used for efficient database interaction with SQL Server, while security mechanisms such as authentication and authorization help maintain data integrity and user privacy.

Overall, the RentAPlace project demonstrates the practical application of modern web technologies in solving real-world problems and provides an efficient, reliable, and user-friendly solution for online property rental systems.

TABLE OF CONTENTS

ABSTARCT I
INTRODUCTION 1
SYSTEM ANALYSIS 2
REQUIREMENTS 3
SYSTEM DESIGN 4
DATABASE DESIGN 6
SYSTEM MODULES 9
TESTING 12
RESULT 14
CONCLUSION 18

INTRODUCTION

The rapid advancement of digital platforms has significantly changed the way people search for rental properties. With increasing urbanization and mobility, there is a growing demand for online systems that allow users to easily find, compare, and book accommodations without relying on traditional methods such as brokers or offline listings. This creates a need for a reliable, user-friendly, and scalable web application that simplifies the property rental process.

The **RentAPlace** project is developed as a web-based rental platform that connects property owners with potential renters for both short-term and long-term stays. The application focuses on providing a seamless experience where users can explore available properties, view detailed information along with images, and make reservations based on their preferences such as location, price, and amenities.

The main objective of this project is to design and implement a full-stack web application that enables users to search and filter properties efficiently while allowing owners to manage their listings effectively. The system includes essential features such as user authentication, property management, reservation handling, and communication between users and property owners, ensuring smooth interaction within the platform.

From the owner's perspective, the application provides complete control over property listings, including adding, updating, and deleting properties, as well as managing reservations and responding to user queries. Additionally, the system supports image uploads and notification features to enhance usability and engagement.

The application is developed using Angular for the frontend and ASP.NET Core Web API for the backend, ensuring a robust and scalable architecture. Entity Framework is used for database operations, enabling efficient data handling and interaction with the SQL Server database. Security mechanisms such as authentication and role-based authorization are implemented to protect user data and ensure secure access.

Overall, the RentAPlace project demonstrates the practical implementation of modern web development technologies and provides an efficient solution for managing property rentals in a digital environment.

SYSTEM ANALYSIS

System Analysis is a crucial phase in the software development life cycle that focuses on understanding system requirements and identifying the needs of different users involved in the application. The **RentAPlace** system is designed to provide a complete online platform for property rental, ensuring smooth interaction between renters and property owners while maintaining efficient system performance.

The system is divided into two main modules: **User Module (Renters)** and **Owner Module**. The **User Module** allows users to register, log in, and explore available properties based on various filters such as location, property type, price, and amenities. Users can view detailed property information along with images, select suitable properties, and make reservations. Additionally, the system enables users to communicate directly with property owners through a messaging feature, enhancing transparency and interaction.

The **Owner Module** provides functionalities for managing property listings efficiently. Owners can add, update, delete, and view their properties through CRUD operations. They can upload property images, respond to user queries, and manage reservation requests. The system also notifies owners when a reservation is made, allowing them to confirm or manage booking statuses effectively.

The application ensures efficient data processing and communication through a well-structured backend developed using ASP.NET Core Web API and a responsive frontend built with Angular. Entity Framework is used for database interaction, enabling smooth and optimized data handling. Swagger is integrated for API testing and documentation, ensuring reliability and ease of development.

Overall, the system is designed to be scalable, secure, and user-friendly, effectively addressing the needs of both renters and property owners while providing a seamless rental experience.

REQUIREMENTS

The requirements of the **RentAPlace** system are categorized into **Functional Requirements** and **Non-Functional Requirements**, ensuring that both system behavior and performance aspects are clearly defined.

Functional Requirements:

1. User Registration and Login for both renters and property owners
2. Search properties based on location, dates, property type, and features
3. View property details along with multiple images
4. Display properties in list or card view
5. Reservation/booking of properties by users
6. Messaging system between users and property owners
7. Owner CRUD operations for managing properties (Add, Update, Delete, View)
8. Upload and manage property images
9. Owner ability to view and respond to user messages
10. Notification system for reservation updates and confirmations

---

Non-Functional Requirements:

1. The system should provide a user-friendly and intuitive interface for easy navigation.
2. The application should be responsive and compatible across multiple devices (desktop, tablet, mobile).
3. The system should ensure data security using authentication and role-based authorization.
4. The application should deliver fast performance with efficient data retrieval and processing.
5. The system should be scalable to support a growing number of users and property listings.
6. The application should maintain high availability and reliability for uninterrupted access.

SYSTEM DESIGN

The System Design of the **RentAPlace** project defines the overall structure of the application and explains how different components interact to deliver a seamless property rental experience. The project follows a **three-layer architecture** consisting of the presentation layer, application/API layer, and database layer. This layered approach ensures that the system remains organized, scalable, and easy to maintain.

The **presentation layer** is developed using Angular. It is responsible for rendering the user interface and handling all client-side interactions. Through this layer, users can register, log in, search for properties, apply filters based on location, price, and amenities, view property details, and make reservations. Property owners can manage their listings, upload images, and respond to user messages through the same interface. The frontend is designed to be responsive and user-friendly, ensuring smooth navigation across different devices.

The **application layer** is developed using ASP.NET Core Web API. This layer acts as an intermediary between the frontend and the database, handling all business logic and processing user requests. It receives HTTP requests from the Angular application, performs necessary validations, executes operations, and returns appropriate responses. Different API controllers are implemented to manage functionalities such as authentication, property management, reservation handling, and messaging. This structured approach helps in maintaining clear separation of concerns.

The **database layer** is responsible for storing all persistent data related to the system. The project uses SQL Server as the database and Entity Framework Core as the ORM tool. Entity Framework simplifies database operations by mapping database tables to C# model classes and handling CRUD operations efficiently. The database stores information related to users, properties, images, reservations, and messages, ensuring organized and consistent data management.

The system is primarily divided into two major modules: the **User Module** and the **Owner Module**. The User Module allows renters to interact with the platform by searching properties, viewing details, making reservations, and communicating with owners. The Owner Module provides functionalities for managing property listings, handling reservations, and responding to user inquiries. This division ensures clarity in roles and efficient management of system functionalities.

The communication flow begins when a user performs an action on the Angular frontend. The frontend sends an HTTP request to the appropriate API endpoint. The ASP.NET Core backend processes the request, interacts with the database through Entity Framework, and returns the result to the frontend. The frontend then updates the user interface accordingly, ensuring smooth and real-time interaction.

The project follows a modular design approach, where each major functionality is implemented as an independent module. For example, authentication, property management, reservation handling, and messaging are developed separately. This

improves code readability, simplifies debugging, and makes the system easier to extend in the future.

Security is an important aspect of the system design. Authentication and role-based authorization are implemented to control access to different features. Sensitive operations such as property management are restricted to authorized owners, ensuring data integrity and secure usage of the application.

Overall, the system design of RentAPlace provides a well-structured and efficient architecture for implementing a full-stack property rental platform. It ensures proper separation of concerns, smooth communication between system components, and reliable handling of both user and owner functionalities.

System Architecture

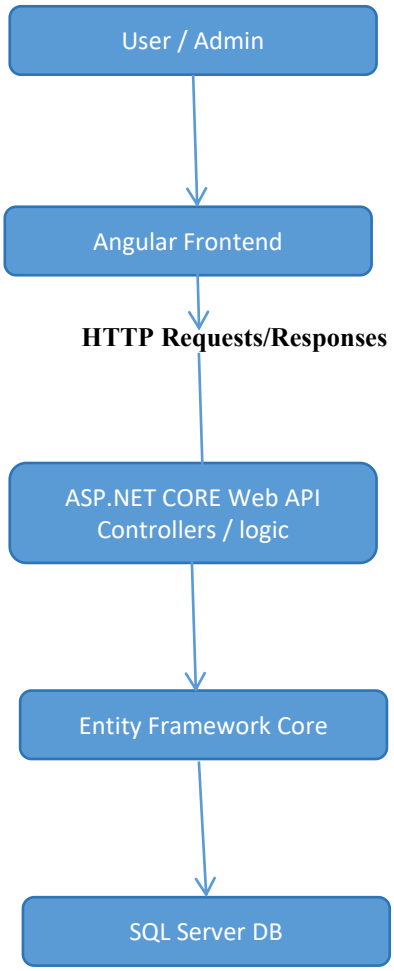


Figure 1: System Architecture of RentAPlace Application

DATABASE DESIGN

The database design of the **RentAPlace** application defines how data is structured, stored, and managed within the system. The application uses **SQL Server** as the relational database management system and **Entity Framework Core** for handling database operations.

The database follows a **relational model**, where data is organized into multiple tables with relationships established using primary keys and foreign keys. This design ensures data integrity, reduces redundancy, and improves overall system performance.

Main Tables in the System

1. Users Table

Stores information about registered users and property owners.

Attributes:

- UserId (Primary Key)
- Name
- Email
- Password
- Role (User/Owner)

2. Properties Table

Stores details of properties listed by owners.

Attributes:

- PropertyId (Primary Key)
- Title
- Description
- Location
- Price
- PropertyType (Flat, Villa, Apartment, etc.)
- OwnerId (Foreign Key)

3. Images Table

Stores images associated with each property.

Attributes:

- ImageId (Primary Key)

- PropertyId (Foreign Key)
- ImageUrl

4. Reservations Table

Stores booking details made by users.

Attributes:

- ReservationId (Primary Key)
- UserId (Foreign Key)
- PropertyId (Foreign Key)
- CheckInDate
- CheckOutDate
- Status (Pending/Confirmed/Cancelled)

5. Messages Table

Stores communication between users and property owners.

Attributes:

- MessageId (Primary Key)
- SenderId (Foreign Key)
- ReceiverId (Foreign Key)
- Content
- Timestamp

Relationships Between Tables

- A **User (Owner)** can have multiple **Properties** (1:N)
- Each **Property** belongs to one **Owner (User)** (N:1)
- A **Property** can have multiple **Images** (1:N)
- Each **Image** is linked to one **Property** (N:1)
- A **User (Renter)** can make multiple **Reservations** (1:N)
- Each **Reservation** is associated with one **User** and one **Property** (N:1)
- A **Property** can have multiple **Reservations** (1:N)
- A **User** can send and receive multiple **Messages** (1:N)
- Each **Message** is associated with a sender and a receiver (N:1)

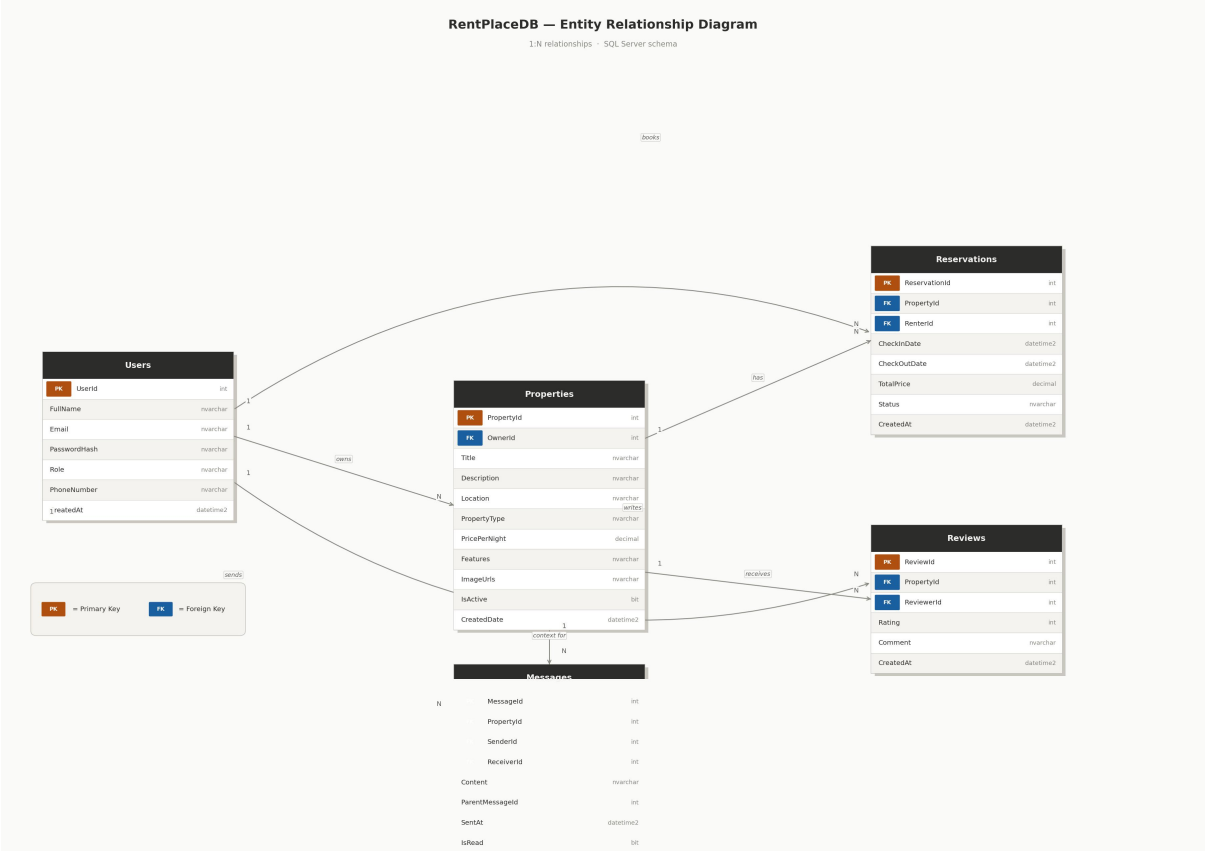
Database Design Advantages

- Ensures **data consistency and integrity** through relational constraints
- Reduces redundancy using normalized table structure
- Improves performance with efficient query handling
- Supports scalability for increasing users and property listings

Storage Optimization Techniques

- Indexing applied on frequently searched fields such as **Location and Price**
- Separation of images into a different table to reduce data duplication
- Use of Entity Framework for optimized database operations
- Proper normalization to avoid unnecessary data repetition

ER Diagram:



SYSTEM MODULES

The **RentAPlace** application is divided into multiple modules to ensure better organization, scalability, and maintainability. Each module is responsible for handling a specific functionality of the system such as user management, property handling, reservations, and communication.

1. User Module

This module manages user registration, login, and authentication for both renters and property owners. It ensures secure access to the system and controls user roles.

Features:

- User Registration
- User Login (JWT Authentication)
- Role Management (User/Owner)

2. Property Module

This module handles property management within the system. Property owners can manage their listings, while users can view and explore available properties.

Features:

- Add Property
- Update Property
- Delete Property
- View Property Listings
- Display Property Details

3. Search & Filter Module

This module allows users to search for properties based on different criteria, improving usability and user experience.

Features:

- Search by Location
- Filter by Property Type
- Filter by Price Range
- Filter by Amenities (e.g., pool, garden, etc.)

4. Image Upload Module

This module manages property images, allowing owners to upload and display images for better visualization.

Features:

- Upload Property Images
- Display Multiple Images
- Store Images in Server Folder

5. Reservation Module

This module manages the booking process for properties. Users can reserve properties, and owners can manage reservation requests.

Features:

- Book/Reserve Property
- View Reservation Details
- Update Reservation Status (Confirm/Cancel)

6. Messaging Module

This module enables communication between users and property owners, improving transparency and interaction.

Features:

- Send Messages
- Receive Messages
- Reply to Messages

7. Notification Module

This module provides updates related to reservations and system activities.

Features:

- Reservation Notification to Owner
- Booking Status Updates
- Email/Alert Notifications

The modular structure of the RentAPlace system ensures that each functionality is handled independently, making the application easier to develop, test, and maintain. It also improves scalability, allowing new features to be added in the future without affecting existing modules.

API ENDPOINTS

The backend APIs for the **RentAPlace** application are developed using **ASP.NET Core Web API**. Swagger is integrated for API testing and documentation, allowing easy visualization and interaction with all available endpoints.

The system provides **RESTful APIs** for different modules such as authentication, property management, reservations, and messaging.

Sample APIs

Authentication

- POST /api/auth/login → User/Owner login
- POST /api/auth/register → User/Owner registration

Property Management

- GET /api/property/all → Fetch all properties
- GET /api/property/{id} → Get property details
- POST /api/property/add → Add new property (Owner)
- PUT /api/property/update/{id} → Update property
- DELETE /api/property/delete/{id} → Delete property

Search & Filter

- GET /api/property/search → Search properties by location, price, type
- GET /api/property/filter → Apply filters on properties

Reservation

- POST /api/reservation/book → Reserve a property
- GET /api/reservation/user → Get user reservations
- GET /api/reservation/owner → Get owner reservations
- PUT /api/reservation/update/{id} → Update reservation status

Messaging

- POST /api/message/send → Send message
- GET /api/message/get/{userId} → Get messages
- POST /api/message/reply → Reply to message

Image Upload

- POST /api/property/upload-image → Upload property image
- GET /api/property/images/{propertyId} → Get property images

TESTING

Testing is an essential phase of the **RentAPlace** project to ensure that all modules function correctly and the system behaves as expected. The application was tested at both frontend and backend levels using manual testing and API testing tools.

Backend API testing was performed using **Swagger UI**. Various endpoints such as authentication, property management, reservation handling, messaging, and image upload were tested with both valid and invalid inputs. This helped verify request processing, response accuracy, and proper error handling.

Frontend testing was carried out by running the Angular application and validating the complete user flow, including user registration, login, property search, viewing property details, booking properties, and sending messages. Owner-side functionalities such as property management, image uploads, and reservation handling were also thoroughly tested.

The testing process confirmed that all modules were properly integrated and the system produced the expected outputs.

Types of Testing

1. Unit/Module Testing

Each module such as User, Property, Reservation, Messaging, and Image Upload was tested individually to ensure correct functionality.

2. Integration Testing

The interaction between Angular frontend, ASP.NET Core Web API backend, and SQL Server database was tested to ensure smooth communication across all layers.

3. API Testing

Swagger UI was used to test API endpoints and verify request/response handling, including JSON responses and status codes.

4. Functional Testing

Core functionalities such as login, registration, property search, booking, and messaging were tested to ensure they work as intended.

5. UI Testing

The frontend interface was tested for proper layout, navigation, responsiveness, and user experience across different devices.

Sample Test Cases

1. User Registration

Input: Valid name, email, password
Expected Result: User account created successfully
Actual Result: Passed

2. User Login

Input: Valid email and password
Expected Result: Login successful and token generated
Actual Result: Passed

3. Property Search

Input: Enter location and filters
Expected Result: Relevant properties displayed
Actual Result: Passed

4. Property Reservation

Input: Select property and booking dates
Expected Result: Reservation created successfully
Actual Result: Passed

5. Image Upload

Input: Upload property images
Expected Result: Images stored and displayed correctly
Actual Result: Passed

6. Messaging

Input: Send message to property owner
Expected Result: Message delivered successfully
Actual Result: Passed

7. Owner Property Management

Input: Add/Edit/Delete property
Expected Result: Property updated in database and UI
Actual Result: Passed

RESULT

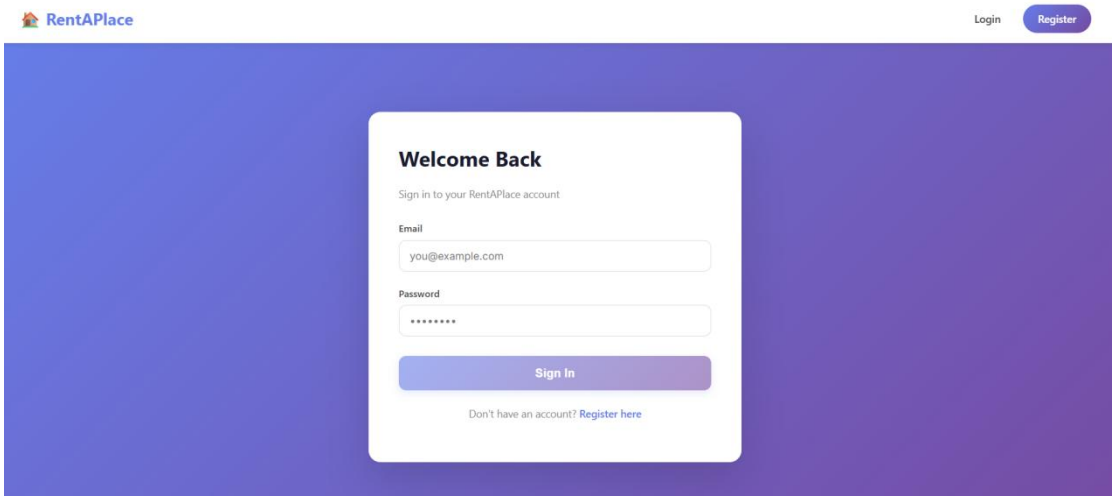
The final output of the **RentAPlace** project demonstrates the successful implementation of a full-stack web application for property rental. The system effectively supports both user-side (renters) and owner-side functionalities. Users can register, log in, search for properties, view detailed listings with images, make reservations, and communicate with property owners. Property owners can manage their listings, upload images, and handle reservation requests efficiently.

The application was successfully integrated with **SQL Server** using **Entity Framework Core**, ensuring smooth data storage and retrieval. Backend APIs were thoroughly tested using **Swagger**, confirming correct request handling and response generation. The frontend developed in Angular interacts seamlessly with the backend APIs, providing a responsive and user-friendly experience.

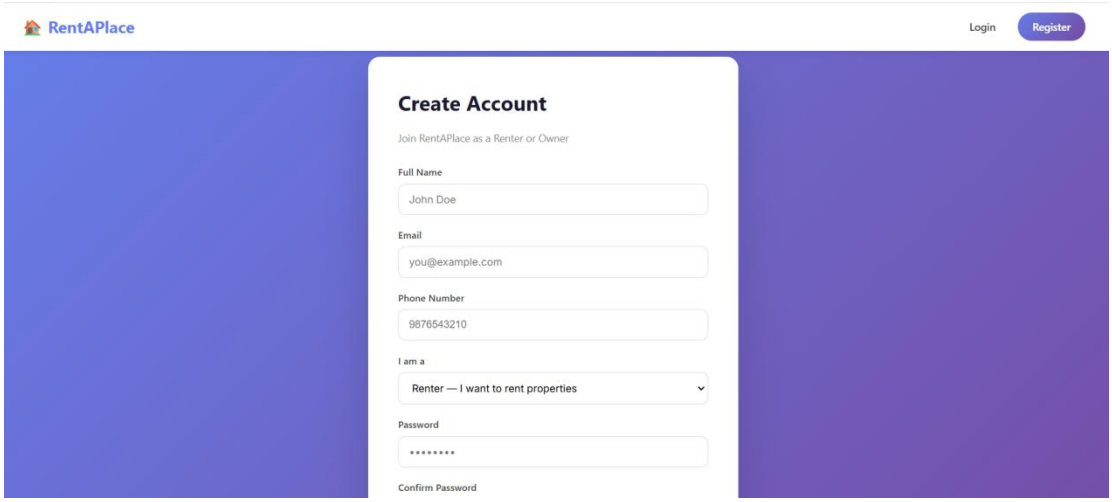
Overall, the system performed as expected, with all modules working correctly and producing accurate results, demonstrating the effectiveness of the design and implementation.

OUTPUT SCREENS

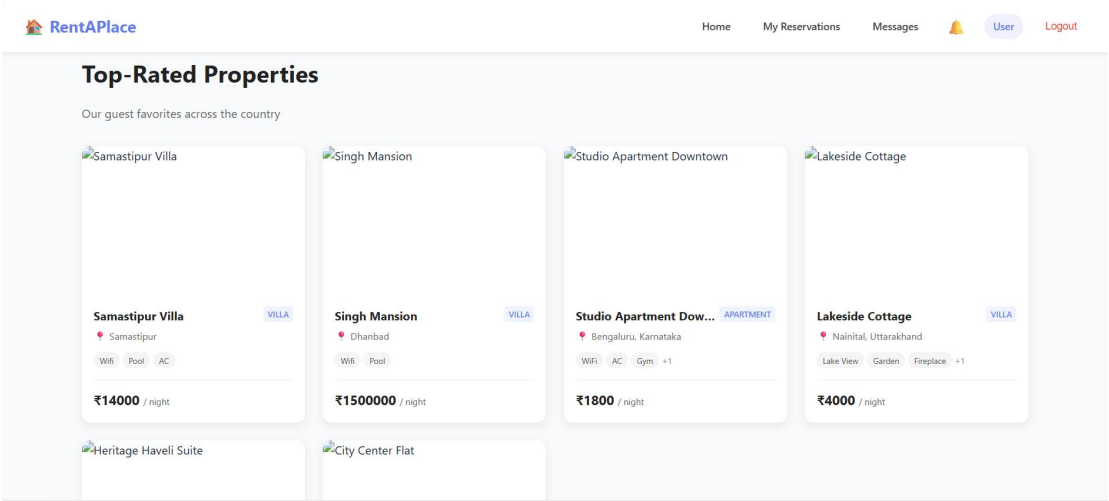
◆ Login Page



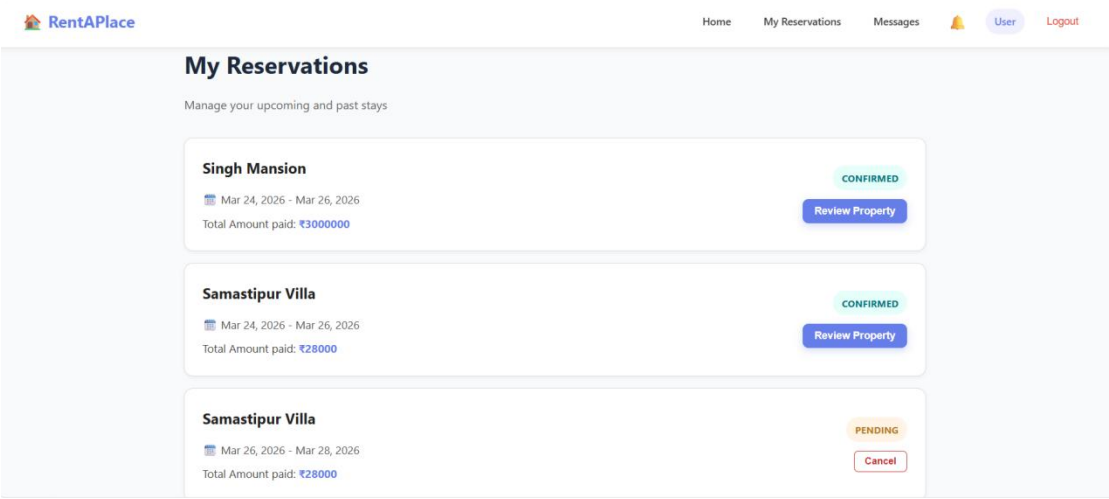
◆ Register Page



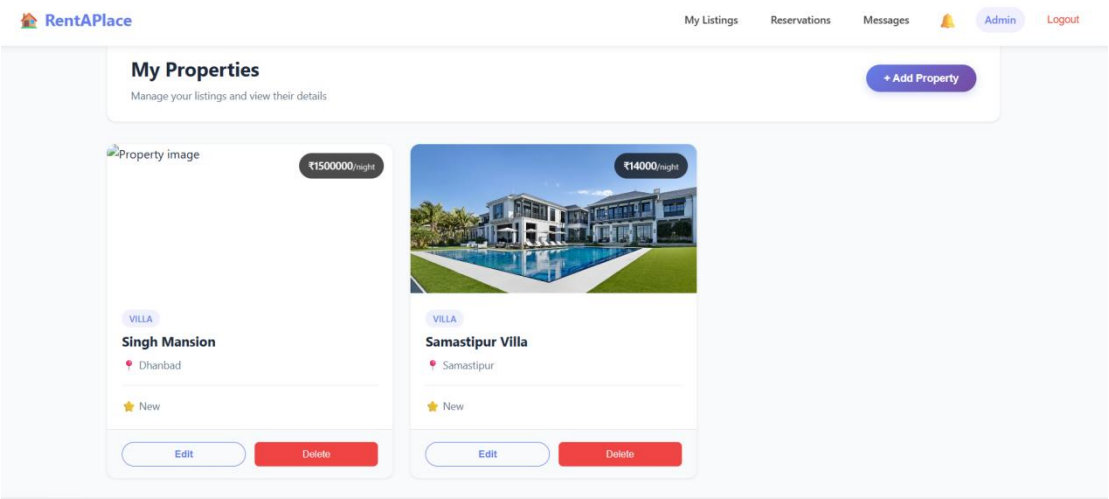
◆ Product Listing Page



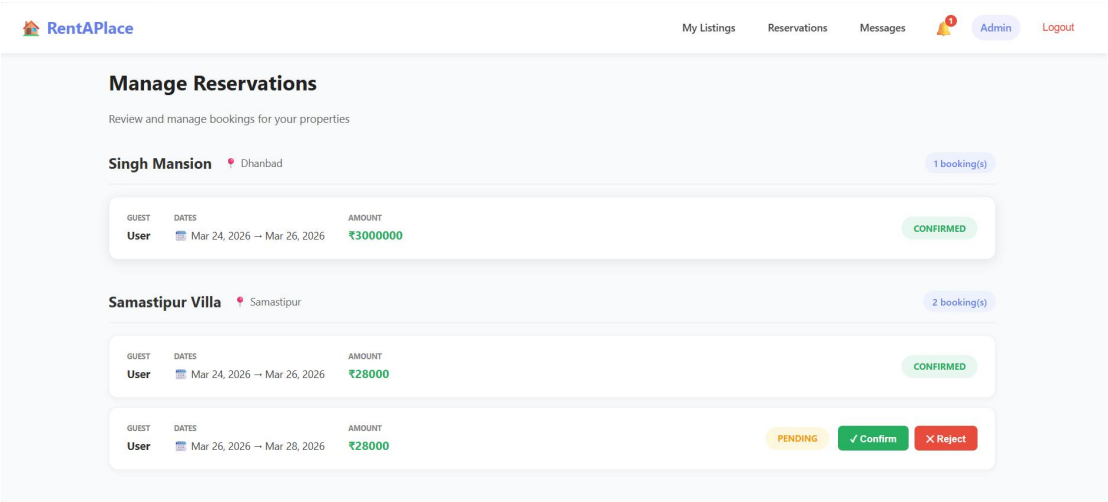
◆ Reservation page



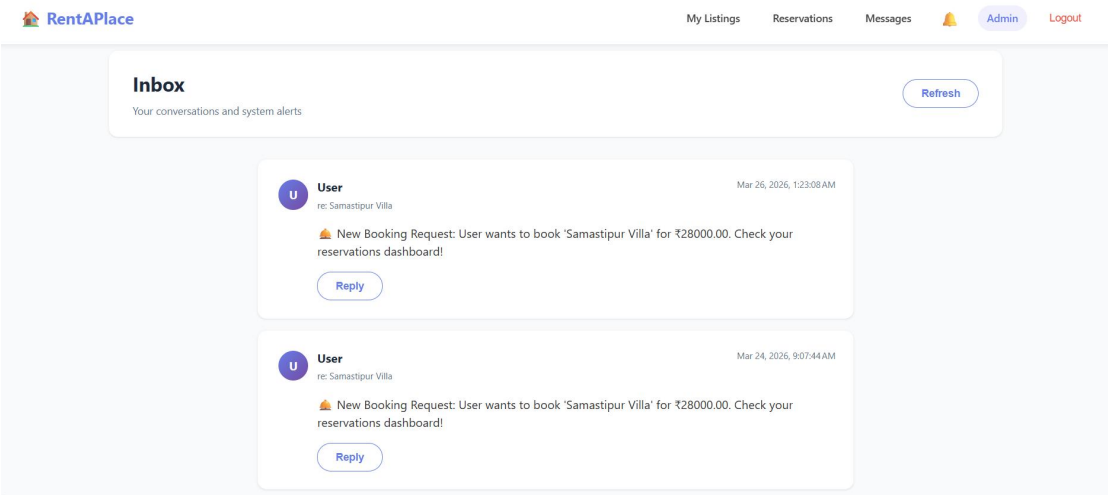
◆ Admin Property Management



◆ Admin Booking Management



◆ Admin Message Management



CONCLUSION

The **RentAPlace** project successfully demonstrates the development of a full-stack web application for property rental using Angular, ASP.NET Core Web API, Entity Framework Core, and SQL Server. The application provides essential functionalities such as user authentication, property management, property search and filtering, reservation handling, image uploads, and messaging between users and property owners.

The system is designed using a modular and layered architecture, ensuring scalability, maintainability, and efficient performance. The integration between the frontend and backend is achieved through RESTful APIs, and Swagger is used for API testing and validation.

This project helped in understanding real-world application development concepts, including database design, API development, frontend-backend integration, and system testing. Overall, the project successfully meets its objective of building a reliable, efficient, and user-friendly online property rental platform.

FUTURE SCOPE:

The **RentAPlace** application can be further enhanced by adding advanced features to improve user experience and system capabilities. Integration of online payment gateways such as Razorpay or Stripe can enable secure and real-time booking transactions.

Additional features such as email notifications, booking confirmations, and real-time status updates can be implemented to enhance communication between users and property owners. The system can also include user reviews and ratings for properties to improve transparency and trust.

Advanced features like map integration using Google Maps can help users locate properties more easily. Recommendation systems based on user preferences can also be implemented to provide personalized property suggestions.

The application can be optimized for mobile platforms by developing a dedicated mobile app. Deployment on cloud platforms such as Azure or AWS can further improve scalability, availability, and performance. An admin dashboard with analytics and reporting features can also be introduced to monitor system activity and improve decision-making.